

Demandas da semana - Projeto FIT PULSE

Período: 19/04 a 27/04

Equipe: Diego: Gerente,

Gustavo: Dev. BackEnd,

Lívia: Dev. FrontEnd,

Gilles: Teste,

Guilherme: Auxiliar.

Prioridades da Semana:

1. Controle de acesso
2. Registro de frequência e heatmap
3. Regra de inadimplência pós-renovação
4. Relatório de ocupação por modalidade

Demandas Definidas:

1.1. Controle de Acesso - Estrutura de Dados[Estudante]

Responsável: Gustavo.

Descrição: Preparar a estrutura necessária para controle de acesso dos estudantes com base em status e pagamento.

- Verificar campo de status do usuário (active, blocked, delinquent)
- Garantir campo de status de pagamento (paid, pending)
- Verificar campo de data de renovação (renewed_at)
- Mapear relação entre status, pagamento e acesso

Entrega esperada: Estrutura de dados pronta para finalizar regras de acesso.

Prazo: 20/04(segunda) - 23:59

1.2. Controle de Acesso - Lógica e Ponto de Acesso(Endpoint)

Responsável: Gustavo e Guilherme.

Descrição: Implementar bloqueio e desbloqueio de acesso.

- POST /access/block
- Validar acesso apenas para estudantes
- Alteração de status do usuário pelo gerente (quando necessário)
- Integrar com inadimplência

Entrega esperada: Controle de acesso funcional aplicado corretamente aos estudantes.

Prazo: 20/04(segunda) - 23:59

1.3. Interface: Controle de Acesso

Responsável: Livia e Gilles.

Descrição: Construir interface consumindo o endpoint de controle de acesso.

- Estudante: exibir status atual (active, blocked, pending) com feedback visual claro
- Gerente: listagem de usuários com ações de Bloquear / Alterar status
- Integração direta com POST /access/block

Entrega esperada: Interface funcional com ações reais e feedback correto de estado.

Prazo: 21/04(terça-feira) - 23:59

2.1. Frequência(heatmap) - Estrutura de Dados[Gerente/Estudante]

Responsável: Gustavo.

Descrição: Preparar estrutura de dados para registro de frequência dos estudantes e suporte à análise de padrões de uso do sistema (horários de pico e dias mais movimentados).

- Criar tabela **frequencies**
- Campos:
 - User_id ou Students_id
 - created_at (timestamp da presença)
- Garantir relacionamento com usuário
- Garantir precisão temporal para análise

Entrega esperada: Banco de dados preparado para registrar frequência e suportar geração de heatmap.

Prazo: 21/04 (terça-feira) — 23:59

2.2. Frequência(heatmap) - Lógica e Ponto de Acesso(Endpoint)

Responsável: Gustavo e Guilherme.

Descrição: Implementar o fluxo de registro de frequência e a geração de dados agregados para análise de uso do sistema.

- Criar Ponto de Acesso **POST /frequency/register**
- Validar se o usuário é do tipo estudante
- Validar se o usuário possui acesso liberado (não bloqueado/inadimplente)
- Registrar presença com timestamp automático (created_at)
- Permitir múltiplos registros por dia
- Criar Ponto de Acesso **GET /reports/frequency/heatmap**
- Agrupar registros por dia da semana e hora
- Contabilizar quantidade de registros por período

Entrega esperada: Registro de frequência funcional e endpoint retornando dados agregados para heatmap.

Prazo: 22/04 (quarta-feira) — 23:59

2.3. Interface: Registro de Frequência

Responsável: Livia e Gilles.

Descrição: Construir primeiro a parte essencial: registro de presença.

- Botão “Registrar Presença”
- Feedback imediato após ação
- Integração com POST /frequency/register

Validação:

- Garantir registro correto no banco
- Testar múltiplos registros no mesmo dia
- Validar bloqueio quando usuário sem acesso tenta registrar

Entrega esperada: Fluxo de presença funcional e confiável.

Prazo: 23/04 (quinta-feira) — 23:59

2.4. Interface: Frequência(heatmap)

Responsável: Livia e Gilles.

Descrição: Construir visualização separada baseada nos dados agregados.

- Heatmap com intensidade por cor
- Eixo X: dias da semana

- Eixo Y: horários
- Consumo de GET /reports/frequency/heatmap

Validação:

- Conferir correspondência entre dados e visualização
- Testar cenários com baixa e alta frequência

Entrega esperada: Visualização funcional e coerente com os dados do sistema.

Prazo: 23/04 (quinta-feira) — 23:59

3.1. Relatório de Inadimplência e churn - Estrutura de Dados[Gerente]

Responsável: Gustavo

Descrição: Preparar e validar a estrutura de dados necessária para identificar estudantes inadimplentes e analisar churn, permitindo distinguir entre usuários que não pagam, que cancelaram ou que deixaram de utilizar o sistema.

- Campo status pagamento
- Campo status usuário
- Campo cancelled_at
- Critério de inatividade (ex: 30 dias sem frequência)

Entrega esperada: Banco de dados estruturado para suportar análise de inadimplência e identificação de churn com base em comportamento e status do usuário.

Prazo: 23/04 (quinta-feira) — 23:59

3.2. Relatório de Inadimplência e Churn - Lógica e Ponto de Acesso(Endpoint)

Responsável: Gustavo e Guilherme

Descrição: Implementar a lógica de identificação e classificação de estudantes com base em comportamento financeiro e uso do sistema, permitindo análise de inadimplência e churn.

- Criar Ponto de Acesso **GET /reports/users/delinquency**

Usuários inadimplentes com:

- status de pagamento = **pending**
- plano ainda vigente ou recentemente renovado

Usuários cancelados com:

- status = **cancelled**
- campo **cancelled_at** preenchido

Usuários inativos com:

- status = **active**
- sem registros de frequência por um período definido (ex: 30 dias)

Entrega esperada: Endpoint funcional retornando estudantes classificados corretamente em inadimplentes, cancelados e inativos, permitindo análise de risco e retenção.

Prazo: 24/04(sexta-feira) 23:59

3.3. Interface: Relatório de Inadimplência e Churn

Responsável: Livia e Gilles.

Descrição: Construir tela de relatório com divisão clara de estados.

- Blocos: Inadimplentes, Cancelados, Inativos
- Exibir nome do usuário e status
- Consumo de GET /reports/users/delinquency

Validação:

- Conferir classificação correta dos usuários
- Validar atualização conforme mudança de estado

Entrega esperada: Relatório funcional com dados confiáveis e bem organizados.

Prazo: 25/04(sábado) 23:59

4.1. Relatório de Ocupação por Modalidade - Estrutura de Dados

Responsável: Gustavo

Descrição: Preparar e validar a estrutura de dados necessária para permitir o agrupamento de estudantes por modalidade de plano, viabilizando a análise de ocupação do sistema.

- Garantir relação entre usuário e plano (user ↔ plan)
- Validar existência dos tipos/modalidades de plano
- Garantir que cada usuário possua um plano ativo associado
- Verificar consistência dos dados para agrupamento (sem duplicidade ou ausência de vínculo)

Entrega esperada: Estrutura de dados validada e consistente para suportar o cálculo de ocupação por modalidade.

Prazo: 25/04(sábado) 23:59

4.2. Relatório de Ocupação por Modalidade - Lógica e Ponto de Acesso

Responsável: Gustavo e Guilherme.

Descrição: Implementar lógica para cálculo da ocupação por modalidade de plano.

- GET /reports/plans/occupation
- Agrupar usuários por tipo de plano
- Retornar quantidade por modalidade

Entrega esperada: Endpoint funcional de ocupação.

Prazo: 25/04(sábado) 23:59

4.3. Interface: Relatório de Ocupação por Modalidade

Responsável: Livia e Gilles.

Descrição: Criar visualização simples da distribuição de usuários por plano.

- Exibir quantidade por modalidade
- Pode ser lista ou gráfico simples
- Consumo de GET /reports/plans/occupation

Validação:

- Conferir contagem correta por plano
- Testar consistência com base de usuários

Entrega esperada: Relatório claro e funcional para análise de ocupação.

Prazo: 26/04(domingo) 23:59

5. Casos de Uso: Regra de Inadimplência Pós-Renovação de Planos[Estudante]

Responsável: Gustavo e Guilherme

Descrição: Implementar regra automática de bloqueio por atraso de pagamento após renovação.

- Após renovação → status pending
- Permitir acesso por 1 dia até pagamento
- Após prazo → bloquear acesso
- Integrar com controle de acesso
- Direcionamento para o pagamento após a renovação, se não for feito é dado o prazo após sair

Entrega esperada: Regra automática aplicada corretamente.

Prazo: 26/04(domingo) 23:59